בקרת איכות ובדיקת תוכנה

שיעור 1 – מבוא לבדיקות תוכנה

Software Testing & Quality Assurance

מה נלמד בקורס?

שבועות	נושא	במה נעסוק
2–1 · היום	**בדיקות תוכנה – הגדרות ועקרונות**	מה זו בדיקה, מה זו איכות, מהו באג, ואילו עקרונות מנחים כל בודק.
5–3	**בדיקות מרכזיות**	סוגי הבדיקות העיקריים שמבצעים בפרויקט תוכנה ומתי משתמשים בכל אחד.
6	**עקרונות Mayer לבדיקות תוכנה**	גישה קלאסית לבדיקות, שמתמקדת בבדיקה כחיפוש מכוון אחר שגיאות.
8–7	**בדיקות יחידה, קופסה שחורה וקופסה פתוחה**	בדיקת רכיב בודד בקוד, ובדיקה עם או בלי הסתכלות על הקוד הפנימי.
9	**גישות בפיתוח בדיקות והבטחת איכות**	איך משלבים בדיקות ואיכות בתהליך הפיתוח עצמו.
14–10	**תכנון וביצוע הבדיקות**	כתיבת תוכנית בדיקות, מקרי בדיקה, הרצה ודיווח על תקלות.

למה בכלל צריך לבדוק תוכנה?

1

משתמש נכנס לאתר קניות ובוחר מוצר.

100 ש״ח

המחיר שמוצג לו בעגלה. הוא משלם.

1,000 ש״ח

הסכום שבו חויב בפועל.

שאלות לכיתה

1. מה השתבש?
2. מי יכול היה לגלות את התקלה?
3. באיזה שלב היה כדאי לזהות אותה?
4. מה הנזק האפשרי לחברה וללקוח?

המקרה: תשובות אפשריות

מה השתבש?

ייתכן שהמחיר נשמר באגורות ונקרא כשקלים, שנוסף אפס בהמרה, או שהמחיר בעגלה שונה מהמחיר שנשלח לחברת האשראי.

מי יכול היה לגלות?

בודק תוכנה שמשווה בין המחיר המוצג לסכום החיוב, המתכנת עצמו בבדיקה שלו, או סקירת קוד של עמית.

מתי כדאי לזהות?

לפני שהאתר עולה לאוויר. ככל שמגלים מוקדם יותר, התיקון פשוט וזול יותר – ואף לקוח לא נפגע.

מה הנזק?

ללקוח: הפסד כספי ותסכול. לחברה: החזרים, עומס על שירות הלקוחות, פגיעה במוניטין ואולי תביעות.

המסקנה: בדיקה נועדה למצוא את הבעיה לפני שהמשתמש מוצא אותה.

דוגמאות לתקלות תוכנה מהחיים

מערכת	התקלה	ההשפעה	למה חשוב לגלות
אפליקציית בנק	מציגה יתרה שגויה	הלקוח מוציא כסף שאין לו ונכנס למינוס	כסף אמיתי ואמון הלקוח בבנק
הרשמה לקורסים	סטודנט נרשם פעמיים לאותו קורס	תופס שני מקומות; סטודנט אחר לא מתקבל	פגיעה בהוגנות ובנתוני המכללה
אתר קניות	הנחה של 20% מחושבת כ־2%	לקוחות משלמים יותר מהמובטח ונוטשים	הפסד מכירות ותלונות
מסך התחברות	כניסה מתאפשרת גם עם סיסמה שגויה	כל אחד יכול להיכנס לחשבון של אחר	פרצת אבטחה ודליפת מידע אישי

מושגי יסוד בבדיקות תוכנה

בדיקת תוכנה · איכות תוכנה · QA ו־Testing

מהי בדיקת תוכנה? (Software Testing)

במילים פשוטות

מפעילים את התוכנה בצורה מתוכננת, ובודקים אם היא מתנהגת כמו שהיא אמורה להתנהג.

הגדרה מקצועית

בדיקת תוכנה היא תהליך של הערכת מוצר תוכנה, כדי לגלות פגמים ולבדוק אם המוצר עומד בדרישות שהוגדרו לו.

דוגמה

מערכת ציונים צריכה לחשב ממוצע. מזינים ציונים 80, 90 ו־100. הממוצע הנכון הוא 90. אם המערכת מציגה 90 – הבדיקה עברה. אם היא מציגה 270 – מצאנו תקלה.

מה מטרת הבדיקות ומה אנחנו מחפשים?

01

לגלות פגמים

למצוא מקומות שבהם התוכנה לא עושה את מה שהיא צריכה – לפני שהמשתמש מגיע אליהם.

02

לבדוק התאמה לדרישות

לוודא שהמערכת עושה את מה שהוגדר: מחשבת נכון, מציגה נכון, חוסמת מה שצריך לחסום.

03

לספק מידע להחלטה

לתת לצוות תמונה על מצב המוצר: האם הוא מוכן לשחרור, ואילו סיכונים נשארו.

מה מחפשים בפועל: חישובים שגויים, פעולות שלא מתבצעות, הודעות שגויות, קריסות, איטיות, ופרצות שמאפשרות פעולה אסורה.

בדיקה (Testing) לעומת תיקון תקלה (Debugging)

	בדיקה – Testing	ניפוי ותיקון – Debugging
מה עושים	מגלים שיש בעיה ומתעדים אותה	מאתרים את הסיבה בקוד ומתקנים
מי עושה בדרך כלל	בודק תוכנה (וגם מפתחים)	המפתח
התוצר	דיווח: מה עשיתי, מה ציפיתי, מה קרה	קוד מתוקן
דוגמה	"לחיצה על 'שלם' עם עגלה ריקה מקריסה את האתר"	המפתח מוסיף בדיקה שהעגלה אינה ריקה

אחרי התיקון חוזרים לבדוק – כדי לוודא שהבעיה נפתרה ושלא נוצרה בעיה חדשה.

דוגמה: בדיקת טופס הרשמה לאתר

הטופס

שם מלא

דוא״ל

סיסמה

הרשמה

מה בודק התוכנה בודק?

פרטים תקינים ← נוצר חשבון ומופיעה הודעת הצלחה

דוא״ל בלי @ ← ההרשמה נחסמת

שדה חובה ריק ← ההרשמה נחסמת

דוא״ל שכבר רשום ← לא נוצר חשבון כפול

לחיצה כפולה על ״הרשמה״ ← נוצר חשבון אחד בלבד

גם אם כל אלה עברו – ייתכנו מצבים שלא בדקנו.

מהי איכות תוכנה? (Software Quality)

תוכנה איכותית היא תוכנה שעונה על הדרישות ועל צורכי המשתמשים – לא רק כזו שאין בה שגיאות חישוב.

פונקציונליות נכונה

Functionality · המערכת עושה את מה שצריך. מחשבון מחבר נכון.

אמינות

Reliability · עובדת לאורך זמן בלי לקרוס. אתר הרשמה שמחזיק ביום פתיחת ההרשמה.

נוחות שימוש

Usability · קל להבין ולהשתמש. כספומט עם הוראות ברורות.

ביצועים

Performance · מגיבה מהר. חיפוש מוצר שנטען בשנייה.

אבטחה

Security · מגינה על מידע. אי אפשר לראות ציונים של סטודנט אחר.

התאמה לדרישות

Requirements · כוללת את מה שהלקוח ביקש. חנות שמאפשרת תשלום בביט, כפי שהוגדר.

כשהתוצאה נכונה והאיכות נמוכה

דוגמה 1 · ביצועים

אפליקציית משלוחים מציגה את התפריט הנכון ומחשבת את המחיר הנכון – אבל כל מסך נטען 25 שניות.

אין שגיאה בחישוב, ובכל זאת משתמשים יעברו לאפליקציה מתחרה.

דוגמה 2 · נוחות שימוש

מערכת ציונים מציגה ציונים נכונים – אבל כדי להגיע אליהם צריך לעבור שבעה מסכים ואין כפתור חזרה.

הנתונים מדויקים, אבל המערכת מתסכלת ולא נוחה.

איכות נמדדת גם באיך המערכת עובדת – לא רק במה היא מחשבת.

הבטחת איכות (QA) ובדיקות תוכנה (Testing)

Quality Assurance

הבטחת איכות

כל הפעילויות שמטרתן **למנוע** בעיות: הגדרת תהליכי עבודה, כללי כתיבת קוד, סקירת דרישות, הדרכת הצוות.

שואלת: האם אנחנו עובדים בצורה שתוביל למוצר טוב?

Software Testing

בדיקות תוכנה

פעילות שמטרתה **להעריך את המוצר**: להפעיל אותו, להשוות לדרישות ולגלות פגמים קיימים.

שואלת: האם המוצר שבנינו עובד כנדרש?

הבדיקות הן חלק מעולם האיכות. QA רחב יותר: הוא עוסק גם בתהליך, לא רק במוצר.

QA לעומת Testing: דוגמת אתר קניות

	QA – הבטחת איכות	Testing – בדיקות
מיקוד	התהליך	המוצר
מטרה	למנוע פגמים	לגלות פגמים
מתי	לאורך כל הפרויקט	כשיש משהו לבדוק: דרישה, קוד, מסך
באתר קניות	קובעים שכל שינוי בחישוב מחיר עובר סקירת קוד	מוסיפים 3 מוצרים לעגלה ובודקים שהסכום נכון
דוגמה נוספת	כותבים רשימת בדיקה לכל דרישה חדשה	מזינים קוד קופון פג תוקף ובודקים שנדחה

תרגול: QA, Testing או שניהם?

א. הצוות קובע שכל קוד חדש ייבדק על ידי מפתח נוסף לפני שמשלבים אותו.

ב. בודקת מזינה סיסמה שגויה במסך ההתחברות ובודקת שמופיעה הודעת שגיאה.

ג. לפני הפיתוח, הצוות קורא את מסמך הדרישות ומחפש דרישות לא ברורות.

ד. מריצים את טופס התשלום עם 100 משתמשים בו־זמנית ומודדים זמני תגובה.

פתרון: QA, Testing או שניהם?

	תשובה	נימוק
א.	**QA**	קביעת כלל עבודה לכל הצוות – שינוי בתהליך שמטרתו למנוע פגמים.
ב.	**Testing**	מפעילים את המוצר ומשווים את התוצאה לתוצאה הצפויה.
ג.	**שניהם**	בודקים מסמך כדי לגלות בו בעיות (Testing), ובכך מונעים פגמים בקוד שעוד לא נכתב (QA).
ד.	**Testing**	מעריכים את המוצר – הפעם מאפיין הביצועים ולא נכונות החישוב.

מה שקובע הוא מטרת הפעולה: לשפר את התהליך ולמנוע, או להעריך את המוצר ולגלות.

שגיאות, פגמים, כשלים ודרישות

Bug · Error · Defect · Failure · Expected / Actual · Requirements

מהו באג? (Bug / Defect)

באג (Bug) הוא כינוי יומיומי לתקלה בתוכנה. המונח המקצועי הוא **פגם (Defect)**: מקום במוצר שבו התוכנה לא בנויה כפי שהיא אמורה להיות.

מזהים פגם כשההתנהגות בפועל שונה מההתנהגות הרצויה.

דוגמה: מחשבון

5 + 3 = 9

תוצאה רצויה	**8**
תוצאה בפועל	**9**

הן שונות ← מצאנו באג.

Error, Defect ו־Failure: שלושה מושגים שונים

Error

שגיאה · אצל האדם

טעות שאדם עושה: מתכנת, מנתח מערכות או מי שכתב את הדרישה.

מתכנת הבין לא נכון את הנוסחה, או הקליד סימן לא נכון.

Defect / Bug

פגם · בתוך המוצר

התוצאה של השגיאה: משהו לא נכון בקוד, בעיצוב או במסמך.

בשורת הקוד כתוב חיבור במקום כפל.

Failure

כשל · בזמן ריצה

מה שרואים כשהמערכת פועלת: התנהגות שונה מהתנהגות הרצויה.

הלקוח רואה מחיר שגוי במסך התשלום.

האדם טועה ← הקוד פגום ← המערכת נכשלת.

דוגמה מתמשכת: חישוב מחיר הזמנה

הדרישה: סכום ההזמנה = מחיר יחידה × כמות. לקוח מזמין 3 חולצות ב־50 ש״ח כל אחת.

Error · 1

המתכנת מקליד בטעות + במקום ×.

←

Defect · 2

בקוד נשמרת השורה:

```
total = price + quantity
```

←

Failure · 3

הלקוח מזמין ורואה לתשלום:

53 ש״ח

במקום 150 ש״ח

הסבר השורה: total הוא הסכום לתשלום, price הוא מחיר יחידה (50), quantity הוא הכמות (3). במקום 50 × 3 = 150 המחשב מחשב 50 + 3 = 53.

פגם לא תמיד מוביל לכשל

אותו קוד פגום, קלט אחר

לקוח מזמין 2 מחזיקי מפתחות ב־2 ש״ח כל אחד.

2 × 2 = 4 · 2 + 2 = 4

התוצאה נכונה במקרה. הפגם קיים, אבל הכשל לא נראה.

קוד שלא מורץ

פגם בחישוב הנחה למועדון לקוחות. אם אף לקוח עוד לא הצטרף למועדון – הקוד הזה לא רץ.

הפגם יושב בקוד ומחכה. הכשל יופיע ביום שהלקוח הראשון יצטרף.

לכן בוחרים קלטים שונים בכוונה – כדי לחשוף פגמים שקלט אחד היה מסתיר.

תוצאה צפויה (Expected) ותוצאה בפועל (Actual)

Expected Result	Actual Result
תוצאה צפויה	**תוצאה בפועל**
מה המערכת **אמורה** לעשות, לפי הדרישה. כותבים אותה **לפני** שמריצים.	מה המערכת **עשתה בפועל** כשהרצנו אותה. כותבים אותה **אחרי** ההרצה.
צפויה = בפועל	**הבדיקה עברה (Pass)**
צפויה ≠ בפועל	**הבדיקה נכשלה (Fail) – מדווחים**

השוואת תוצאות: שלוש דוגמאות

מה עשינו	תוצאה צפויה	תוצאה בפועל	סטטוס
כספומט: משיכת 200 ש״ח מחשבון עם 500 ש״ח	יוצאים 200 ש״ח, היתרה 300 ש״ח	יוצאים 200 ש״ח, היתרה 300 ש״ח	**Pass**
מערכת ציונים: ממוצע של 70 ו־90	80	160	**Fail**
מסך התחברות: סיסמה שגויה	הודעת שגיאה, נשארים במסך	כניסה לאזור האישי	**Fail**

בלי תוצאה צפויה אי אפשר לקבוע אם 160 הוא באג. רק הדרישה אומרת לנו מה נכון.

דרישות תוכנה (Requirements)

מהי דרישה?

תיאור כתוב של מה שהמערכת צריכה לעשות, או של תכונה שהיא צריכה לקיים. הדרישה נכתבת לפני הפיתוח, בשיתוף הלקוח.

למה בודקים לפיה?

הדרישה קובעת את התוצאה הצפויה. בלעדיה כל בודק יחליט לבד מה "נכון", וכל אחד יחליט אחרת.

שתי דוגמאות לדרישות

"המערכת תאפשר למשתמש להתחבר באמצעות שם משתמש וסיסמה תקינים."

"סטודנט לא יוכל להירשם לקורס שבו כל המקומות תפוסים."

איך הופכים דרישה לבדיקה?

”המערכת תאפשר למשתמש להתחבר באמצעות שם משתמש וסיסמה תקינים.

1 · מה הפעולה?

התחברות: הזנת שם משתמש וסיסמה ולחיצה על ”כניסה”.

2 · אילו קלטים?

פרטים תקינים; סיסמה שגויה; משתמש שלא קיים.

3 · מה צפוי?

תקינים ← כניסה. השאר ← אין כניסה.

הדרישה לא אומרת מה יוצג כשאין כניסה – הודעה? איזו? זו שאלה שבודק מעלה לפני הפיתוח.

דרישות ברורות ודרישות לא ברורות

לא ברורה – אי אפשר לבדוק	ברורה – אפשר לבדוק
"האתר ייטען מהר."	"דף הבית ייטען תוך 2 שניות לכל היותר."
"הסיסמה תהיה חזקה."	"הסיסמה תכיל לפחות 8 תווים, מהם לפחות ספרה אחת."
"אפשר להזמין כמות סבירה של כרטיסים."	"אפשר להזמין בין 1 ל־8 כרטיסים בהזמנה."

מבחן פשוט: האם שני בודקים שונים יכתבו לאותה דרישה את אותה תוצאה צפויה?

תרגיל: בדיקת כתובת דוא״ל בהרשמה

הדרישה

מערכת הרשמה מאפשרת להירשם עם כתובת דוא״ל. כתובת הדוא״ל חייבת להכיל סימן @.

---

1. מה יקרה אם הכתובת תקינה?
2. מה יקרה אם אין בה @?
3. איך אפשר לדעת שהמערכת פועלת נכון?

פתרון: בדיקת כתובת דוא״ל

קלט	תוצאה צפויה (לפי הדרישה)	תוצאה בפועל (לדוגמה)	סטטוס
dana@gmail.com	ההרשמה מתקבלת	ההרשמה התקבלה	**Pass**
danagmail.com	ההרשמה נדחית	ההרשמה התקבלה	**Fail**
dana@	ההרשמה מתקבלת – יש @	ההרשמה התקבלה	**Pass**
שדה ריק	ההרשמה נדחית – אין @	ההרשמה נדחתה	**Pass**

איך יודעים שהמערכת פועלת נכון?

משווים כל תוצאה בפועל לתוצאה הצפויה שנקבעה מראש. שורה 3 עוברת, אבל מראה שהדרישה חלשה: ״dana@״ אינה כתובת אמיתית. את זה מדווחים כבעיה בדרישה, לא כבאג בקוד.

עקרונות בסיסיים בבדיקות תוכנה

שבעה עקרונות שמנחים כל בודק

שבעה עקרונות בסיסיים

1. בדיקות מגלות נוכחות של פגמים
2. לא ניתן לבדוק את כל האפשרויות
3. כדאי להתחיל לבדוק מוקדם
4. ריכוז פגמים
5. פרדוקס חומר ההדברה
6. בדיקות תלויות בהקשר
7. היעדר שגיאות אינו מבטיח הצלחה

אלה עקרונות כלליים המקובלים בעולם התוכנה. עקרונות Mayer הם נושא נפרד – שבוע 6.

עיקרון 1: בדיקות מגלות נוכחות של פגמים

בדיקות יכולות להראות שיש פגמים. הן לא יכולות להוכיח שאין.

הסבר

כל בדיקה בודקת מצב אחד. בדיקה שעברה אומרת שבמצב הזה המערכת עבדה – ולא יותר. מצב שלא בדקנו נשאר לא ידוע.

דוגמה

בדקנו מסך התחברות ב־10 מקרים, וכולם עברו. לא בדקנו סיסמה עם רווח, שם משתמש באותיות עבריות, או 50 ניסיונות כושלים ברצף. כל אחד מהם עלול לחשוף באג.

עיקרון 1: טעות נפוצה ומסקנה

טעות נפוצה

”כל הבדיקות עברו, אז אין באגים במערכת.”

ניסוח נכון

”במקרים שבדקנו לא נמצאו באגים.”

מסקנה

בדיקות מקטינות את הסיכון שיישארו פגמים – אבל לא מורידות אותו לאפס.

עיקרון 2: לא ניתן לבדוק את כל האפשרויות

בדיקה של כל קלט וכל מצב (Exhaustive Testing) אינה אפשרית בפועל.

94

תווים אפשריים בכל מקום בסיסמה (אותיות, ספרות, סימנים)

≈6×10^15

שילובים לסיסמה באורך 8 תווים בלבד

190,000

שנה, בקצב של 1,000 בדיקות בשנייה

דוגמה נוספת

הזמנת כרטיסים: 1–8 כרטיסים × עשרות סרטים × קודי הנחה × שעות הקרנה. כל שדה נוסף מכפיל את מספר השילובים.

עיקרון 2: טעות נפוצה ומסקנה

טעות נפוצה

"בודק טוב בודק הכול."

ניסוח נכון

"בודק טוב בוחר מה לבדוק: לפי הדרישות, לפי הסיכון, ולפי מה שחשוב למשתמש."

איך בוחרים, בשלב הזה

מקרה תקין אחד לפחות לכל דרישה

מקרים לא תקינים שהמערכת צריכה לחסום

ערכים בקצה הטווח – למשל בדיוק 8 תווים (נרחיב בשבועות 7–8)

עיקרון 3: כדאי להתחיל לבדוק מוקדם

בדיקות מתחילות כבר בשלב הדרישות והתכנון – לא רק כשהקוד מוכן.

מתגלה לפני הפיתוח

בדרישה כתוב: "סטודנטים יקבלו הנחה". הבודק שואל: כמה אחוז? איך מוכיחים שאתה סטודנט?

תיקון: משפט אחד במסמך ופגישה של 10 דקות.

מתגלה אחרי שהמערכת מוכנה

המפתח הניח 5% בלי אימות. הלקוח התכוון ל־15% עם תעודת סטודנט.

תיקון: שינוי קוד, מסך חדש להעלאת תעודה, ובדיקה מחדש של כל תהליך התשלום.

עיקרון 3: טעות נפוצה ומסקנה

טעות נפוצה

”אין מה לבדוק עד שיש קוד שרץ.”

ניסוח נכון

”אפשר לבדוק דרישות ותכנון עוד לפני שנכתבה שורת קוד אחת.”

מסקנה

ככל שבעיה מתגלה מוקדם יותר, יש פחות עבודה שצריך לשנות בעקבותיה – ולכן התיקון זול יותר.

עיקרון 4: ריכוז פגמים (Defect Clustering)

רוב הפגמים נמצאים בדרך כלל בחלק קטן מהמערכת.

למה?

אזורים מסוימים מורכבים יותר: יש בהם הרבה חוקים, הם מתחברים למערכות חיצוניות, או שמשנים אותם לעתים קרובות. בכל אלה קל יותר לטעות.

מסך תשלום	דף "אודות"
מחיר, הנחות, קופונים, סוגי כרטיסי אשראי, חיבור לחברת האשראי	טקסט ותמונה קבועים, בלי חישובים ובלי קלט מהמשתמש

עיקרון 4: טעות נפוצה ומסקנה

טעות נפוצה

”מצאנו ותיקנו 5 באגים במסך התשלום, אז עכשיו הוא נקי.”

ניסוח נכון

”במקום שבו מצאנו באגים יש סיכוי גבוה למצוא עוד – כדאי להמשיך לבדוק אותו.”

מסקנה

משקיעים יותר בדיקות באזורים מורכבים ובאזורים שבהם כבר נמצאו פגמים – בלי לוותר לגמרי על השאר.

עיקרון 5: פרדוקס חומר ההדברה (Pesticide Paradox)

חזרה על אותן בדיקות בדיוק מפסיקה עם הזמן לגלות פגמים חדשים.

הסבר

כמו חרקים שמפתחים עמידות לחומר הדברה קבוע. הבדיקות כבר מצאו את מה שהן מסוגלות למצוא, והבאגים האלה תוקנו. פגמים במקומות אחרים נשארים מחוץ לטווח שלהן.

דוגמה

חנות מקוונת עוברת את אותן 5 בדיקות כל שבוע, כבר חצי שנה – כולן עוברות. בינתיים נוספה אפשרות לשלם בביט. אף אחת מ־5 הבדיקות לא נוגעת בה.

עיקרון 5: טעות נפוצה ומסקנה

טעות נפוצה

"הבדיקות עוברות כבר חצי שנה – המערכת יציבה."

ניסוח נכון

"ייתכן שהבדיקות שלנו כבר לא מכסות את מה שהשתנה במערכת."

מסקנה

ממשיכים להריץ בדיקות קיימות, אבל גם מעדכנים אותן ומוסיפים בדיקות חדשות לכל שינוי ותכונה חדשה.

עיקרון 6: בדיקות תלויות בהקשר

אופן הבדיקה נקבע לפי סוג המערכת ולפי חומרת הנזק אם היא תיכשל.

	משחק מחשב	מערכת בנקאית	מערכת רפואית
מה הכי חשוב	חוויה, גרפיקה, מהירות תגובה	דיוק בחישובים, אבטחה	בטיחות המטופל, אמינות
נזק מכשל	שחקן מתוסכל	הפסד כספי, פגיעה באמון	פגיעה בבריאות ובחיים
עומק הבדיקה	דגש על שימוש וחוויה	כל חישוב והרשאה נבדקים	הכי מחמיר, לפי תקנים מחייבים

עיקרון 6: טעות נפוצה ומסקנה

טעות נפוצה

"יש דרך אחת נכונה לבדוק תוכנה, ומשתמשים בה בכל פרויקט."

ניסוח נכון

"מתאימים את הבדיקות לסוג המערכת, למשתמשים ולסיכון."

דוגמה נוספת

טעות בתאריך באתר מועדון הסטודנטים – מביכה. טעות בציון במערכת הציונים – עלולה לפגוע בזכאות לתואר.

מסקנה

לפני שמתכננים בדיקות שואלים: מה המערכת עושה, ומה יקרה אם היא תיכשל?

עיקרון 7: היעדר שגיאות אינו מבטיח הצלחה

מערכת בלי פגמים ידועים יכולה עדיין להיכשל – אם היא לא עונה על צורכי המשתמשים.

דוגמה 1

אפליקציית ציונים חדשה: התחברות מהירה, עיצוב יפה, אפס קריסות. אבל אי אפשר לראות בה את הציון הסופי בקורס – הדבר שבשבילו הסטודנטים פותחים אותה.

דוגמה 2

אתר הזמנת מונית שעובר את כל הבדיקות – אבל דורש 12 שדות חובה לפני כל נסיעה. המשתמשים חוזרים להזמין בטלפון.

עיקרון 7: טעות נפוצה ומסקנה

טעות נפוצה

”אם אין באגים, הלקוח יהיה מרוצה.”

ניסוח נכון

”המערכת צריכה גם לעבוד נכון וגם לעשות את מה שהמשתמשים צריכים.”

מסקנה

בודקים מול הדרישות – וגם שואלים אם הדרישות עצמן משקפות את מה שהמשתמשים באמת צריכים.

תרגול מעשי: אתם הבודקים

בדיקת מערכת התחברות לאתר המכללה

התרחיש: מערכת התחברות לאתר המכללה

תיאור המערכת

סטודנט מתחבר לאזור האישי באמצעות מספר תעודת זהות וסיסמה.

חשבון קיים לצורך התרגיל

ת״ז: 123456789

סיסמה: Negev2026 (9 תווים)

דרישות המערכת

1. תעודת הזהות חייבת להכיל בדיוק 9 ספרות.
2. הסיסמה חייבת להכיל לפחות 8 תווים.
3. כאשר פרטי ההתחברות תקינים ומתאימים לחשבון קיים, המשתמש יועבר לאזור האישי.
4. כאשר הפרטים אינם תקינים, תופיע הודעת שגיאה.
5. שדות ריקים אינם מאפשרים התחברות.

משימה: הציעו לפחות 8 מקרי בדיקה

מקרה בדיקה (Test Case) הוא תיאור של בדיקה אחת: מה בודקים, עם איזה קלט, ומה התוצאה הצפויה.

#	מה בודקים?	קלט	תוצאה צפויה
1			
2			
3			
4			
5			
6			
7			
8			

פתרון: מקרי בדיקה 1–5

#	מה בודקים	קלט (ת״ז / סיסמה)	תוצאה צפויה	דרישה
1	פרטים תקינים	123456789 / Negev2026	**מעבר לאזור האישי**	1, 2, 3
2	סיסמה קצרה מדי (7 תווים)	123456789 / Negev20	אין כניסה; הודעת שגיאה	2, 4
3	ת״ז קצרה מדי (8 ספרות)	12345678 / Negev2026	אין כניסה; הודעת שגיאה	1, 4
4	ת״ז ארוכה מדי (10 ספרות)	1234567890 / Negev2026	אין כניסה; הודעת שגיאה	1, 4
5	אותיות בשדה ת״ז	12345678A / Negev2026	אין כניסה; הודעת שגיאה	1, 4

נוסח הודעת השגיאה לא מוגדר בדרישות – לכן בודקים רק שהודעת שגיאה מופיעה, לא מה כתוב בה.

פתרון: מקרי בדיקה 6–10

#	מה בודקים	קלט (ת״ז / סיסמה)	תוצאה צפויה	דרישה
6	סיסמה ריקה	(ריק) / 123456789	אין כניסה. הודעה – לא מוגדר במפורש	5
7	ת״ז ריקה	Negev2026 / (ריק)	אין כניסה. הודעה – לא מוגדר במפורש	5
8	שני השדות ריקים	(ריק) / (ריק)	אין כניסה. הודעה – לא מוגדר במפורש	5
9	סיסמה באורך תקין אך שגויה	123456789 / Wrong2026	אין כניסה; הודעת שגיאה	3, 4
10	ת״ז תקינה שאינה שייכת לחשבון	987654321 / Negev2026	אין כניסה; הודעת שגיאה	3, 4

בבדיקות 9–10 הפרטים תקינים בפורמט אבל לא מתאימים לחשבון. דרישה 4 מדברת על פרטים ״לא תקינים״ – ומקובל לפרש זאת גם כפרטים שגויים. כדאי לאשר את הפרשנות מול מי שכתב את הדרישה.

מה הדרישות לא מגדירות

כשהדרישה לא קובעת התנהגות – לא ממציאים אותה. מתעדים את השאלה ומבררים מול מי שכתב את הדרישות.

מה נוסח הודעת השגיאה? האם היא אומרת איזה שדה שגוי?

האם יש אורך מקסימלי לסיסמה?

מה קורה עם רווח לפני או אחרי הקלט?

האם הסיסמה רגישה לאותיות גדולות וקטנות?

בשדה ריק: מופיעה הודעה, או שכפתור הכניסה פשוט לא פעיל?

האם בודקים ספרת ביקורת של תעודת הזהות?

האם יש חסימה אחרי מספר ניסיונות כושלים?

"לפחות 8 תווים" – אילו תווים מותרים?

דיון: מה למדנו מהתרגיל?

1. האם בדקנו את כל האפשרויות?
2. אילו בדיקות נוספות אפשר להוסיף?
3. אילו דרישות אינן מספיק ברורות?
4. האם אפשר לקבוע בוודאות שהמערכת תקינה?

דיון: תשובות

1 · בדקנו הכול?

לא. בדקנו 10 מקרים מתוך מספר עצום של קלטים אפשריים. בחרנו מקרים מייצגים לכל דרישה (עיקרון 2).

2 · בדיקות נוספות

סיסמה של 8 תווים בדיוק; ת״ז עם מקף או רווח; סיסמה נכונה באותיות קטנות; ניסיונות כושלים רבים ברצף; סיסמה ארוכה מאוד.

3 · דרישות לא ברורות

נוסח ההודעה, טיפול בשדות ריקים, אורך מקסימלי, תווים מותרים, חסימה. כדאי לברר לפני הפיתוח (עיקרון 3).

4 · תקינה בוודאות?

לא. אפשר לומר רק: ״ב־10 המקרים שבדקנו המערכת התנהגה לפי הדרישות״ (עיקרון 1).

מה למדנו היום

בדיקת תוכנה – הפעלה מתוכננת של המערכת והשוואה לדרישות, כדי לגלות פגמים.

איכות – לא רק חישוב נכון: גם אמינות, נוחות, ביצועים ואבטחה.

QA מונע דרך תהליכים; **Testing** מעריך את המוצר.

Failure ← Defect ← Error – אדם טועה, הקוד פגום, המערכת נכשלת.

Expected / Actual – קובעים מראש מה צפוי, ומשווים למה שקרה.

דרישה – מקור התוצאה הצפויה. דרישה לא ברורה היא ממצא.

מקרה בדיקה – מה בודקים, קלט, ותוצאה צפויה.

7 עקרונות – בדיקה היא בחירה מושכלת, לא ניסיון לבדוק הכול.

מילון מושגים (1 מתוך 2)

מונח באנגלית	בעברית	הסבר קצר
Software Testing	בדיקת תוכנה	הערכת המוצר כדי לגלות פגמים ולבדוק התאמה לדרישות
Quality Assurance (QA)	הבטחת איכות	פעילויות ותהליכים שמטרתם למנוע פגמים
Software Quality	איכות תוכנה	מידת ההתאמה לדרישות ולצורכי המשתמשים
Error	שגיאה	טעות אנושית
Defect / Bug	פגם / באג	משהו שגוי בתוך המוצר (קוד, מסמך, עיצוב)
Failure	כשל	התנהגות שגויה שנראית בזמן הפעלת המערכת
Debugging	ניפוי ותיקון	איתור הסיבה לפגם בקוד ותיקונה

מילון מושגים (2 מתוך 2)

מונח באנגלית	בעברית	הסבר קצר
Requirement	דרישה	תיאור כתוב של מה שהמערכת צריכה לעשות
Expected Result	תוצאה צפויה	מה המערכת אמורה לעשות, לפי הדרישה
Actual Result	תוצאה בפועל	מה המערכת עשתה כשהרצנו אותה
Test Case	מקרה בדיקה	בדיקה אחת: מה בודקים, קלט ותוצאה צפויה
Pass / Fail	עבר / נכשל	תוצאת ההשוואה בין הצפוי לבפועל
Exhaustive Testing	בדיקה ממצה	בדיקה של כל האפשרויות – לא אפשרית בפועל

עבודה 1 – זיהוי תקלות ותכנון בדיקות בסיסיות

עבודה 1: אתר להזמנת כרטיסים לקולנוע

המערכת מאפשרת

בחירת סרט

בחירת כמות כרטיסים בין 1 ל־8

חישוב מחיר ההזמנה

הזנת קוד הנחה

ביצוע הזמנה

דרישות

1. מחיר כרטיס רגיל הוא 40 ש״ח.
2. ניתן להזמין בין 1 ל־8 כרטיסים בהזמנה.
3. אי אפשר להזמין אפס כרטיסים או כמות שלילית.
4. קוד הנחה MOVIE10 מעניק 10% הנחה מסכום הכרטיסים.
5. קוד הנחה אחר אינו מעניק הנחה.
6. בסיום הזמנה תקינה תוצג הודעת אישור.
7. אם כמות הכרטיסים אינה חוקית, לא ניתן להשלים את ההזמנה.

עבודה 1: מבנה, ניקוד והגשה

20
שאלה 1 · מושגי יסוד
ארבעה סעיפי הסבר במילים שלכם

20
שאלה 2 · זיהוי באגים
ארבעה מצבים: תואם או לא, ומה צפוי

40
שאלה 3 · תכנון בדיקות
טבלה של 8 מקרי בדיקה לפחות

20
שאלה 4 · חשיבה ביקורתית
האם 8 בדיקות מספיקות? ושתי דרישות חדשות

דרישות הגשה

- קובץ PDF או Word
- עבודה אישית
- ציון 0–100
- תשובות ברורות ומסודרות
- נימוק לכל תשובה
- טבלאות קריאות ומסודרות

הנוסח המלא של השאלות נמצא בדף העבודה.